

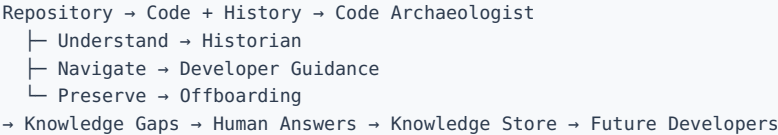
Technical Build Specification

Scope, stack, and the technical rules everyone builds against.

Read this if: engineers (backend and frontend); PM/design can skim for scope only

One rule above all others: nobody changes the architecture, tech stack, API contracts, database design, agent responsibilities, or core scope solo. Get sign-off from the Team Lead and update the docs — otherwise we integrate five different projects instead of one.

Product lifecycle



In scope

- Public GitHub repositories only
- Repository acquisition and read-only analysis (never executed)
- Multi-language support: Python, JavaScript, TypeScript, Java, C++, Go
- Supabase Auth; user-isolated repository and knowledge data
- Repository explorer, Monaco code viewer, history/timeline views, architecture visualization
- Historian, Developer Guidance, Knowledge Gap, and Offboarding agents
- Deterministic risk analyzer
- Evidence classification and confidence scoring
- Human knowledge capture and reuse
- Guardrails against off-topic questions and prompt injection
- A fully deployed web application

Out of scope

IDE plugin, private repositories, code execution of any kind, automated code modification, arbitrary shell execution, installing packages from analyzed repositories, enterprise SSO/RBAC, enterprise org management, claims of universal language support.

Technology stack

Layer	Choices
Backend	Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, pgvector, LangGraph, Groq
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Monaco Editor, React Flow
Infrastructure	GitHub, Supabase Auth, Supabase PostgreSQL + pgvector, Render, Vercel

Free tiers only for the hackathon build.

Evidence model

Every meaningful answer is classified as one of: **Verified** (backed directly by repository evidence), **Inferred** (a reasonable conclusion the evidence doesn't state outright), or **Human knowledge** (supplied by a person). Confidence (High/Medium/Low) sits on top of this classification, not instead of it.

Guardrails

Repository content is untrusted data, never an instruction. The system must never execute repository code, run arbitrary shell commands, or install packages from the analyzed repo. Secrets stay server-side. Tools are fixed and whitelisted. Off-topic questions and prompt-injection attempts get rejected — every answer stays tied to the repository's own context.

Example: "Why does this repo use JWT here?" → answered normally. "Suggest medicine for the flu" → refused: "this assistant only answers questions related to the analyzed repository."

Definition of done

A module is done when it: follows this spec, the API Contract, and the Database Design; has no hard-coded secrets; has basic error handling and tests; works with the shared data structures; has a README or docstrings where needed; and has been merged into

develop successfully.

Build priorities

- P0 – Repository import & analysis, FastAPI + DB, Historian, Developer Guidance, Code explorer, Risk, Knowledge Gap, Auth, Offboarding, Multi-language foundation
- P1 – Architecture visualization, Evidence/confidence polish, UI polish

If time runs short, cut implementation depth before cutting the product thesis — a thin version of the whole story beats a polished fragment.